GCBF+: A Neural Graph Control Barrier Function Framework for Distributed Safe Multi-Agent Control

Paper Documentation Notes

Prajith S

1/10/2026

1 Overview

Centralized training, distributed execution.

- *Offline (GPU server):* collect data, solve the QP targets, update network weights.
- *Online:* each robot carries its own lightweight policy π_ϕ and reads only local sensors (LiDAR rays, relative position/velocity of neighbours within radius R). **No inter-agent communication** at runtime.

Why no game theory. All agents are fully cooperative: they run the same π_ϕ and share one objective (reach goal while keeping $h_\theta \geq 0$), so there are no competing payoffs and no equilibrium to solve. Coordination (who yields) emerges from training instead:

- *Shared-policy training:* the loss for agent i backpropagates through the actions of neighbours $j \in \tilde{\mathcal{N}}_i$.
- *Implicit alignment:* each agent knows neighbours run the same policy, so it anticipates their yielding/steering using only local observations.

2 Problem Formulation

N agents, indexed by $\mathcal{V}_a := \{1, \ldots, N\}$, must reach goals in an obstacle environment without collisions. Each follows control-affine dynamics

$$\dot{x}_i = f(x_i) + g(x_i)\, u_i, \qquad x_i \in \mathcal{X},\ u_i \in \mathcal{U}, \tag{1}$$

(same $f, g, \mathcal{X}, \mathcal{U}$ for all agents—a homogeneity assumption). f and g are assumed to be locally Lipschitz continuous. Stacked: $\bar{x} := [x_1; \ldots; x_N] \in \mathcal{X}^N$, $\bar{u} := [u_1; \ldots; u_N] \in \mathcal{U}^N$, giving $\dot{\bar{x}} = \bar{f}(\bar{x}) + \bar{g}(\bar{x})\bar{u}$.

Geometry and sensing.

- Workspace $P \subset \mathbb{R}^n$ $(n = 2, 3)$; $p_i \in P$ is the first n entries of x_i; goal $p_i^g \in P$.
- Each agent has n_{rays} evenly spaced LiDAR rays, max range R. Ray j of agent i is $y_j^{(i)} \in \mathcal{X}$ (first n entries = hit point $p_j^{(i)}$, rest zero-padded); $\bar{y}_i := [y_1^{(i)}; \ldots; y_{n_{\text{rays}}}^{(i)}]$.
- Each agent fits in a ball of radius r $(R > 2r > 0)$: agent–agent safety needs $\|p_i - p_j\| > 2r$; obstacle safety needs $\left\|y_j^{(i)}\right\| > r$ for all rays.

Problem 1. Design a distributed policy π_i such that for every agent i:

- **Safety (obstacles):** $\left\|y_j^{(i)}(t)\right\| > r \ \forall j,\ t \geq 0.$
- **Safety (agents):** $\|p_i(t) - p_j(t)\| > 2r \ \forall j \neq i,\ t \geq 0.$
- **Liveness:** $\inf_{t\geq 0} \|p_i(t) - p_i^g\| = 0$ i.e., each agent eventually reaches its goal location

Approach

Assume a *nominal controller* π_{nom} that gives liveness but not necessarily safety, and build a GCBF-based distributed policy that adds safety.

3 GCBF: A Safety Certificate for Multi-Agent Systems

3.1 CBF review: S vs. C

For $\dot{x} = F(x, u)$: $S \subset \mathcal{X}$ is the **physical safe set** (no collision now); $C := \{x : h(x) \geq 0\}$ is the **forward-invariant set** of a CBF h. A valid CBF needs $C \subseteq S$: C trims from S the states from which a future collision is unavoidable. S itself is generally *not* invariant (dynamics and actuation limits), so "safe now" does not imply "can stay safe". If $C \subset S$, existence of a CBF implies existence of a safe control.

3.2 Graph structure

Nodes $V = \mathcal{V}_a \cup \mathcal{V}_o$: agent nodes and obstacle nodes (LiDAR hit points of all agents). **Directed edges** (i, j) carry information from sender j to receiver agent i iff $2r < \|p_i - p_j\| < R$ (note $(i, j) \neq (j, i)$).

Neighbourhood sets.

- $\tilde{\mathcal{N}}_i$: all sender neighbours of i within R (excludes i); its size fluctuates, so it cannot be fed directly to a fixed-size function.
- $M - 1$: max number of neighbours that can fit inside radius R without colliding (given r).
- $\mathcal{N}_i \subseteq \tilde{\mathcal{N}}_i \cup \{i\}$: the M closest nodes *including* i (ties broken by smaller index); if fewer than M, pad with dummy nodes. We need this because the QP constraints are written in this . Note that we DO NOT need it for the GNN The dummies can be conviniently positioned at infinity.

Example: $M - 1 = 4$, $M = 5$

Sparse: 2 neighbours in $R \Rightarrow |\tilde{\mathcal{N}}_i| = 2$; $\mathcal{N}_i$ = agent + 2 neighbours + 2 dummies.
Crowded: 8 neighbours $\Rightarrow |\tilde{\mathcal{N}}_i| = 8$; $\mathcal{N}_i$ = agent + 4 closest.
Remark 1 of the paper: while agents are safe, the crowded case ($|\tilde{\mathcal{N}}_i| > M - 1$) cannot occur, and changes of $\mathcal{N}_i$ happen only without collision at distance R

Local snapshot $\bar{x}_{\mathcal{N}_i}(t) = [x_i; x_{j_1}; \ldots; x_{\text{dummy}}] \in \mathcal{X}^M$, and $\mathcal{N}_i^{<R} := \{j \in \mathcal{N}_i : \|p_i - p_j\| < R\}$ which is just the original set without the dummies.

3.3 Global safe and avoid sets

$$\mathcal{S}_N := \Big\{ \bar{x} \in \mathcal{X}^N \;\Big|\; \big(\big\| y_j^{(i)} \big\| > r,\ \forall i \in \mathcal{V}_a, \forall j \in n_{\text{rays}} \big) \wedge \big(\min_{i \neq j} \|p_i - p_j\| > 2r \big) \Big\}, \qquad \mathcal{A}_N := \mathcal{X}^N \backslash \mathcal{S}_N. \tag{2}$$

These are defined over the *entire* space, not just neighbours inside R.

3.4 Graph Control Barrier Function

Definition 1 (GCBF)

A C^1 function $h : \mathcal{X}^M \to \mathbb{R}$ is a GCBF if there exist an extended class-$\mathcal{K}$ function α and policies $\pi_i : \mathcal{X}^M \to \mathcal{U}$ such that for all $\bar{x} \in \mathcal{X}^N$, $N \geq M$,

$$\dot{h}(\bar{x}_{\mathcal{N}_i}) + \alpha(h(\bar{x}_{\mathcal{N}_i})) \geq 0 \quad \forall i \in \mathcal{V}_a, \qquad \dot{h}(\bar{x}_{\mathcal{N}_i}) = \sum_{j \in \mathcal{N}_i} \frac{\partial h(\bar{x}_{\mathcal{N}_i})}{\partial x_j} (f(x_j) + g(x_j) u_j), \tag{3}$$

with $u_j = \pi_j(\bar{x}_{\mathcal{N}_j})$, and

1. **Zero gradient at the boundary:** $\dfrac{\partial h}{\partial x_j}(\bar{x}_{\mathcal{N}_i}) = 0$ for all $j \in \mathcal{N}_i \setminus \mathcal{N}_i^{<R}$;
2. **Value invariance:** $h(\bar{x}_{\mathcal{N}_i}) = h(\bar{x}_{\mathcal{N}_i^{<R}})$.

Intuition: nodes at distance $\geq R$ contribute nothing to h. Without condition 2, h would jump (step function) as neighbours enter/leave the neighbourhood.

Lemma 1. For a valid GCBF, $t \mapsto h(\bar{x}_{\mathcal{N}_i(t)}(t))$ is continuously differentiable despite jumps in $\mathcal{N}_i(t)$.

3.5 Structural form of the GCBF (Remark 2)

Take

$$h(\bar{x}_{\mathcal{N}_i}) = \xi_1\Big(\sum_{j \in \mathcal{N}_i} w(x_i, x_j)\, \xi_2(x_i, x_j) \Big), \tag{4}$$

with $\xi_2 : \mathcal{X} \times \mathcal{X} \to \mathbb{R}^\rho$ a feature encoder, $\xi_1 : \mathbb{R}^\rho \to \mathbb{R}$ a readout, and $w : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a smooth weighting function (graph attention in practice) satisfying, whenever $\|p_i - p_j\| \geq R$,

$$w(x_i, x_j) = 0, \qquad \frac{\partial w}{\partial x_j}(x_i, x_j) = 0.$$

Proof that (4) satisfies Conditions 1 and 2. Let $s_i := \sum_{k \in \mathcal{N}_i} w(x_i, x_k) \xi_2(x_i, x_k)$, so $h = \xi_1(s_i)$.

Condition 2. For $k \in \mathcal{N}_i \setminus \mathcal{N}_i^{<R}$, $\|p_i - p_k\| \geq R$ so $w = 0$ and those terms vanish: $s_i = \sum_{k \in \mathcal{N}_i^{<R}} w\, \xi_2 = s_i^{<R}$, hence $h(\bar{x}_{\mathcal{N}_i}) = \xi_1(s_i^{<R}) = h(\bar{x}_{\mathcal{N}_i^{<R}})$.

Condition 1. For such a $j \neq i$, x_j appears only in the $k = j$ term. By the chain and product rules,

$$\frac{\partial h}{\partial x_j} = \frac{\partial \xi_1}{\partial s_i} \Big[\frac{\partial w}{\partial x_j} \xi_2^\top + w \frac{\partial \xi_2}{\partial x_j} \Big] = \frac{\partial \xi_1}{\partial s_i} \big[0 \cdot \xi_2^\top + 0 \cdot \tfrac{\partial \xi_2}{\partial x_j} \big] = 0. \qquad \square$$

3.6 Invariant sets and the safety theorem

$$\mathcal{B}_h := \{\tilde{x} \in \mathcal{X}^M \mid h(\tilde{x}) \geq 0\} \quad \text{(local 0-superlevel set)} \tag{5}$$

$$C_{N,i} := \{\bar{x} \in \mathcal{X}^N \mid \bar{x}_{\mathcal{N}_i} \in \mathcal{B}_h\} \quad \text{(agent } i\text{'s system set)} \tag{6}$$

$$C_N := \textstyle\bigcap_{i=1}^N C_{N,i} = \{\bar{x} \mid h(\bar{x}_{\mathcal{N}_i}) \geq 0 \ \forall i\} \quad \text{(global safe invariant set)} \tag{7}$$

1. **Local sensor input only (z_i):** the agent only needs its own onboard sensors (e.g., relative LiDAR distances and local relative positions of nearby neighbors inside sensing radius R, denoted $\tilde{\mathcal{N}}_i$).
2. **Onboard processing:** agent i feeds z_i into its own onboard chip/microcontroller and executes a single, lightning-fast neural network forward pass to compute its control command u_i.
3. **Zero communication needed:** agent i does not need a central server, nor does it need to ask its neighbors what control actions u_j they are choosing.

Theorem 1 (safety of GCBF)

Let h be a GCBF and $C_N \subset \mathcal{S}_N$ for each $N \in \mathbb{N}$. Then for any N, closed-loop trajectories with $\bar{x}(0) \in C_N$ under any locally Lipschitz $\bar{u} : \mathcal{X}^N \to \mathcal{U}^N_{\text{safe}}$ satisfy $\bar{x}(t) \in \mathcal{S}_N \ \forall t \geq 0$, where $\mathcal{U}^N_{\text{safe}} := \{\bar{u} \in \mathcal{U}^N \mid \dot{h}(\bar{x}_{\mathcal{N}_i}) + \alpha(h(\bar{x}_{\mathcal{N}_i})) \geq 0, \ \forall i\}$.

A single GCBF thus certifies safety for *any* N (the earlier GCBFv0 only certified a specific N), explaining why training on 8 agents carries to 1000+.

The individual $C_{N,i}$ do not all need to be a subset of $\mathcal{S}_N$ as long as the intersection $C_N \subset \mathcal{S}_N$ in Theorem 1. For example, if $\mathcal{S}_N$ can be written as the intersection of sets $\mathcal{S}_{N,i}$, i.e. $\mathcal{S}_N = \bigcap_{i=1}^N \mathcal{S}_{N,i}$, then it is sufficient that $C_{N,i} \subseteq \mathcal{S}_{N,i}$ for all $i \in \mathcal{V}_a$ to obtain $C_N \subseteq \mathcal{S}_N$, since

$$C_N := \bigcap_{i=1}^N C_{N,i} \subseteq \bigcap_{i=1}^N \mathcal{S}_{N,i} =: \mathcal{S}_N$$

4 Safe Control Policy Synthesis

4.1 Prior work (GCBFv0) and its problem

GCBFv0 used a hierarchy: a nominal controller π_{nom} (LQR/PID) for goal reaching, and a neural controller switched in only when safety was threatened. Training added a loss pulling the learned controller toward π_{nom}, which *competes* with the CBF safety loss: the policy either sacrificed safety or got stuck in deadlocks. It also needed an expensive online policy-refinement step with inter-agent communication of control actions. No Actuation Limits were used and the version did work only for specific values of N.

4.2 Minimal-deviation QP

Given $u_i^{\text{nom}} = \pi_{\text{nom}}(x_i, p_i^g)$, α and h, solve centrally

$$\min_{\bar{u}} \sum_{i \in \mathcal{V}_a} \|u_i - u_i^{\text{nom}}\|^2 \tag{17a}$$

$$\text{s.t. } u_i \in \mathcal{U} \quad \forall i \tag{17b}$$

$$\sum_{j \in \mathcal{N}_i} \frac{\partial h}{\partial x_j}(f(x_j) + g(x_j)u_j) \geq -\alpha(h(\bar{x}_{\mathcal{N}_i})) \quad \forall i \tag{17c}$$

This is a QP (strictly convex quadratic objective, polytope $\mathcal{U}$, constraint (17c) linear in $\bar{u}$). Its solution is the policy $\pi_{\text{QP}}(\bar{x})$: the *minimal-deviation safe projection* of the goal-seeking nominal command (which is blind to obstacles).

Why (17) cannot run online.

- *Coupling:* (17c) ties u_i to neighbours' u_j, giving one large problem needing global state or heavy communication every step.
- *Distributed QP (e.g. ADMM)* needs multiple high-rate communication rounds per control step—too slow for 1000+ drones at 50 Hz.

4.3 GCBF+ solution: distil π_{QP} into a distributed π_ϕ

	Offline training (GPU server)	Online execution (onboard)
Swarm size	Small ($N = 8$)	Large ($N = 1000+$)
Information	Central server sees all agents	Local sensors only (radius R)
Compute limit	Seconds/minutes per batch	50 Hz = 20 ms on tiny chips
Solve π_{QP}?	**Yes** (8-agent QP is fast)	**No** (1000-agent QP in 20 ms impossible)

Online, agent i computes $u_i = \pi_\phi(z_i)$ with one forward pass of its own network on local input z_i (relative LiDAR/neighbour data in $\tilde{\mathcal{N}}_i$): no central server and no knowledge of u_j. Thus using just the Local sensor input,Onboard processing and Zero communication with other agent, we train and compute π_ϕ

5 Graph Neural Network Architecture

Nodes: agents, goals, and LiDAR hit points; node feature v_i is a 3-dim one-hot of the of the node. **Edge features** e_{ij}: info from j to i, always including $p_{ij} = p_j - p_i$ (plus relative velocity if dynamics need it). Edge input $z_{ij} = [v_i, v_j, e_{ij}]^\top$; agent input $z_i = [z_{i1}^\top, \ldots, z_{i|\tilde{\mathcal{N}}_i|}^\top]^\top$ (no padding/truncation needed for the GNN).

Attention message passing.

$$q_{ij} = \psi_{\theta_1}(z_{ij}) \quad \text{(encoder)} \tag{9}$$

$$w_{ij} = \text{softmax}_j(\psi_{\theta_2}(q_{ij})) \quad \text{(attention weight = importance of } j \text{ to } i) \tag{10}$$

$$m_{ij} = \psi_{\theta_3}(q_{ij}) \quad \text{(message/value)} \tag{11}$$

$$q_i = \textstyle\sum_{j \in \tilde{\mathcal{N}}_i} w_{ij}\, m_{ij} \quad \text{(neighbourhood summary)} \tag{12}$$

followed by a readout MLP ψ_{θ_4}. The same structure (separate parameters) gives both networks:

- GCBF: $h_i = \psi_{\theta_4}(q_i) \in \mathbb{R}$; policy: $u_i = \pi_\phi(z_i) = \psi_{\phi_4}(q_i) \in \mathcal{U}$ (m outputs).

This realises the form (4) with $\xi_2 \leftrightarrow (\psi_{\theta_1}, \psi_{\theta_3})$, $w \leftrightarrow$ attention, $\xi_1 \leftrightarrow \psi_{\theta_4}$.

Interpretation of the quantities.

- ψ_{θ_i} are MLPs: neural networks in the sense that they consist of multiple linear–activation-function blocks.
- q_{ij}: the embedding of edge (i, j) produced by the encoder.
- w_{ij}: the *importance* of the assigned neighbor j to agent i.
- q_i: what the entire neighborhood of i means to agent i (weighted summation of $m_{ij} = \psi_{\theta_3}(q_{ij})$ is the neighbor aggregation).

Attention-GNN pipeline, shared in structure by h_θ and π_ϕ.

Why attention

Attention weights are what satisfy Definition 1's two conditions. They are *not* hard-coded: after training, w_{ij} and its derivative go to 0 as $d_{ij} \to R$ with no extra supervision (Remark 2, Fig. 1).

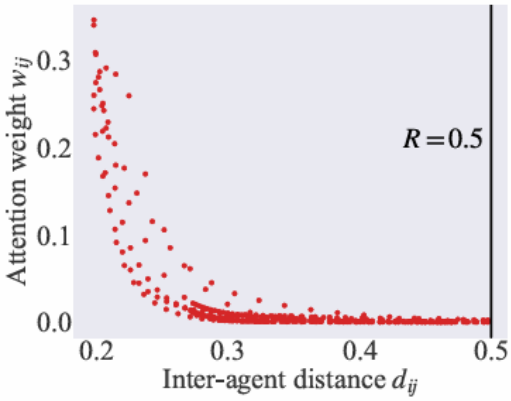


Figure 1: Attention weights w_{ij} vs. inter-agent distance d_{ij} ($R = 0.5$): $w_{ij} \to 0$ as $d_{ij} \to R$, so GCBF+ learns Conditions 1 and 2 automatically.

Layer	GCBF network h_θ (params θ)	Policy network π_ϕ (params ϕ)
Encoder	$\psi_{\theta_1}(z_{ij})$	$\psi_{\phi_1}(z_{ij})$
Attention gate	$\psi_{\theta_2}(q_{ij})$	$\psi_{\phi_2}(q_{ij})$
Attention value	$\psi_{\theta_3}(q_{ij})$	$\psi_{\phi_3}(q_{ij})$
Readout	$\psi_{\theta_4}(q_i) \to h_i \in \mathbb{R}$ (1 scalar)	$\psi_{\phi_4}(q_i) \to u_i \in \mathcal{U}$ (m outputs)

6 GCBF+ Loss Functions

$$\mathcal{L}(\theta, \phi) = \mathcal{L}_{\text{CBF}}(\theta, \phi) + \mathcal{L}_{\text{ctrl}}(\phi), \quad \mathcal{L}_{\text{CBF}} = \sum_{i \in \mathcal{V}_a} \mathcal{L}_{\text{CBF},i}, \quad \mathcal{L}_{\text{ctrl}} = \sum_{i \in \mathcal{V}_a} \mathcal{L}_{\text{ctrl},i}, \tag{13}$$

with $[x]_+ := \max(0, x)$.

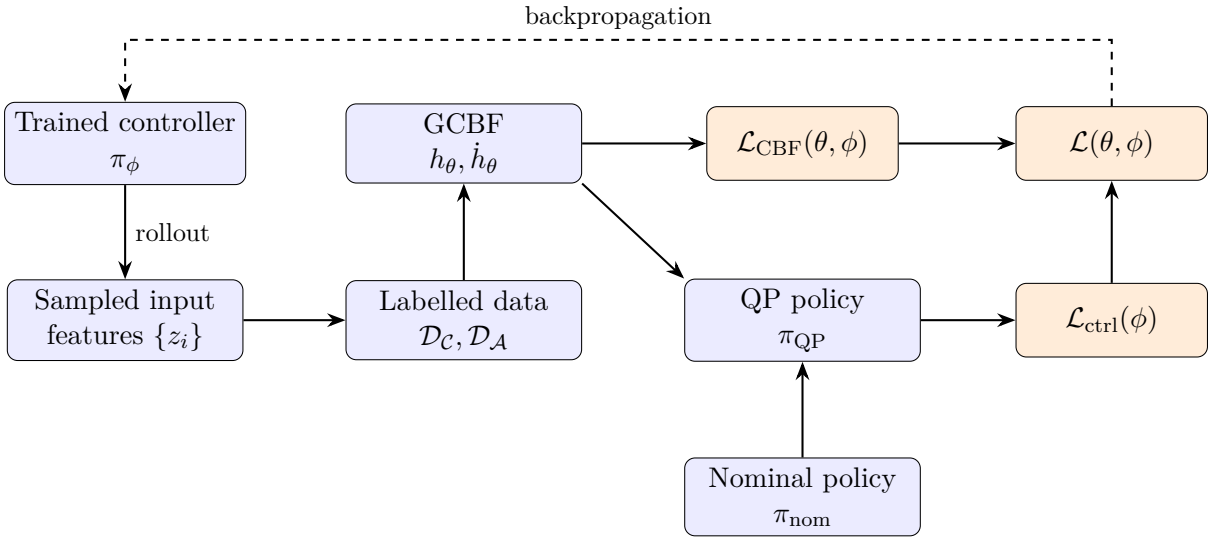


Figure 2: GCBF+ training. Rollouts of the previous π_ϕ label features as safe-invariant ($\mathcal{D}_\mathcal{C}$) or unsafe ($\mathcal{D}_\mathcal{A}$); a CBF-QP with the previous h_θ and π_{nom} gives π_{QP}; π_{QP} and the GCBF conditions define $\mathcal{L}$.

6.1 CBF loss

Hinge (one-sided) rather than square (two-sided) loss:

$$\mathcal{L}_{\text{CBF},i} := \eta_{\text{deriv}} \sum_{z_i} \Big[\gamma - \dot{h}_\theta(z_i) - \alpha(h_\theta(z_i))\Big]_+ + \sum_{z_i \in \mathcal{D}_{\mathcal{C},i}} [\gamma - h_\theta(z_i)]_+ + \sum_{z_i \in \mathcal{D}_{\mathcal{A},i}} [\gamma + h_\theta(z_i)]_+ . \quad (14)$$

It depends on ϕ because the policy determines $\dot{h}_\theta$.

- **Term 1 (derivative):** enforces $\dot{h}_\theta + \alpha(h_\theta) \geq \gamma > 0$, so under π_ϕ the trajectory keeps the barrier non-negative.
- **Term 2 ($\mathcal{D}_{\mathcal{C},i}$, safe-invariant):** enforces $h_\theta \geq \gamma > 0$. With $\gamma = 0.02$: $h = 0.05 \Rightarrow$ penalty 0; $h = -0.10 \Rightarrow$ penalty 0.12.
- **Term 3 ($\mathcal{D}_{\mathcal{A},i}$, collision):** enforces $h_\theta \leq -\gamma < 0$. With $\gamma = 0.02$: $h = -0.05 \Rightarrow$ penalty 0; $h = +0.10 \Rightarrow$ penalty 0.12.

6.2 Estimating $\dot{h}_\theta$; multi-agent backpropagation

$\dot{h}_\theta$ is estimated by finite differences,

$$\dot{h}_\theta(z_i) \approx \frac{h_\theta(z_i(t_{k+1})) - h_\theta(z_i(t_k))}{\delta t}. \quad (15)$$

Neighbours may enter or leave R between steps, which normally causes a jump in h and a huge spike after dividing by δt. Because the learned $w_{ij} \to 0$ smoothly at R, Lemma 1 makes differentiable and the estimate is well behaved.

$z_i(t_{k+1})$ depends on the actions $u_j = \pi_\phi(z_j)$ of agent i *and* its neighbours $j \in \tilde{\mathcal{N}}_i$, so backpropagating $\mathcal{L}_i$ sends gradients into all their policies. This trains neighbours to cooperatively help agent i keep the joint barrier condition satisfied. (Neighbour inputs are *not* needed by π_ϕ at test time.)

6.3 Control loss and π_{QP}

$$\mathcal{L}_{\text{ctrl},i}(\phi) := \eta_{\text{ctrl}} \left\| \pi_\phi(z_i) - \pi_{\text{QP},i}(\bar{x}) \right\|, \tag{16}$$

where $\pi_{\text{QP},i}$ is the i-th component of the QP solution computed with the GCBF from the previous iteration. It updates only ϕ. If the QP is infeasible early in training, a relaxation $\zeta \geq 0$ is added to (17c) with a large penalty in (17a); once $\mathcal{L}_{\text{CBF}}$ converges to zero the QP is feasible on the sampled data.

6.4 Why π_{QP} instead of π_{nom} in the control loss

Prior works (GCBFv0, MDCBF, MARL): $\mathcal{L}_{\text{ctrl}} = \eta_{\text{ctrl}} \|\pi_\phi - \pi_{\text{nom}}\|$. Near an obstacle, π_{nom} says "go straight to goal" while $\mathcal{L}_{\text{CBF}}$ says "steer away", so the terms conflict and η_{ctrl} becomes a balancing act:

- η_{ctrl} too large ⇒ goal reaching dominates, collisions (Fig. 5, top right);
- η_{ctrl} too small ⇒ over-conservative, freezing/deadlock (Fig. 5, top left).

Even at the best η_{ctrl} the learned and nominal inputs do not align, so the total loss cannot reach zero unless π_{nom} is itself safe (e.g. an empty environment): avoiding the obstacle raises $\mathcal{L}_{\text{ctrl}}$, not avoiding it raises $\mathcal{L}_{\text{CBF}}$.

GCBF+: π_{QP} is the closest action to π_{nom} that satisfies all safety constraints, so it is already safe *and* goal-seeking. Matching it drives $\mathcal{L}_{\text{ctrl}}$ to zero while satisfying the CBF conditions, so the losses reinforce each other, the total loss converges to 0, and safety of π_ϕ is not sensitive to η_{ctrl} (no delicate tuning).

Role of η_{ctrl}

η_{ctrl} scales $\mathcal{L}_{\text{ctrl}}$ against the CBF terms (whose derivative term is scaled by η_{deriv}): larger η_{ctrl} ⇒ tracking dominates the gradient; smaller ⇒ safety terms dominate.

6.5 Estimating $\dot{h}_\theta$ and backpropagation

Finite-difference estimate. $\dot{h}_\theta$ is estimated by

$$\dot{h}_\theta(z_i) \approx \frac{h_\theta(z_i(t_{k+1})) - h_\theta(z_i(t_k))}{\delta t}. \tag{17}$$

Neighbours may enter or leave R between steps, which would normally cause a jump in h and a huge spike after dividing by δt. Because the learned $w_{ij} \to 0$ smoothly at R, Lemma 1 makes h_θ C^1 and the estimate is well behaved.

Backpropagation The dashed arrow in the training figure is ordinary gradient computation: $\mathcal{L}(\theta, \phi)$ is differentiated w.r.t. the network weights and used to update them. The loss reaches the policy π_ϕ through two routes:

1. **Directly, via $\mathcal{L}_{\text{ctrl}}$:** $\eta_{\text{ctrl}} \|\pi_\phi(z_i) - \pi_{\text{QP},i}\|$ is an explicit function of ϕ (supervised regression onto the QP action, which acts as a fixed label).
2. **Indirectly, via $\mathcal{L}_{\text{CBF}}$:** the derivative term needs $h_\theta(z_i(t_{k+1}))$, and the state at t_{k+1} is produced by stepping the dynamics with the actions the policy chose. Hence

$$\phi \;\to\; u_j = \pi_\phi(z_j) \;\to\; x_j(t_{k+1}) \;\to\; z_i(t_{k+1}) \;\to\; h_\theta \;\to\; \dot{h}_\theta \;\to\; \mathcal{L}_{\text{CBF}}, \tag{18}$$

and backpropagation applies the chain rule along this chain in reverse. Intuitively, the loss tells the policy: "had you output a slightly different action, $\dot{h}_\theta$ would be closer to

satisfying $\dot{h}_\theta + \alpha(h_\theta) \geq \gamma$". This requires the simulation step to be differentiable (autodiff framework such as JAX/PyTorch).

Gradients through neighbours. $z_i(t_{k+1})$ contains the *next* states of the neighbours $j \in \tilde{\mathcal{N}}_i$, which depend on their actions $\pi_\phi(z_j)$. So $\mathcal{L}_i$ sends gradient into the policy evaluated at each neighbour's input, not only agent i's own. All agents share the same weights ϕ, so there is no separate "neighbour policy" being edited: the same ϕ receives gradient contributions from several places in the computation graph, which add up. The shared policy therefore learns actions that help *others'* barrier conditions as well as its own. This is the only coordination mechanism in GCBF+ (no runtime communication), and it is needed only in training: neighbour inputs are *not* required by π_ϕ at test time.

7 Data Collection and Labelling

Data z_i is split into $\mathcal{D}_{\mathcal{C},i}$ (safe control-invariant) and $\mathcal{D}_{\mathcal{A},i}$ (unsafe/collision). Both are needed because of actuation limits: a state that is not colliding now may still make a collision inevitable (e.g. moving too fast to stop). Labelling it "safe" teaches an unavoidable crash as safe; labelling it "unsafe" is wrong since no collision exists now and a better action might save it.

Control-invariant set

A region of states from which at least one valid control keeps the robot safe. The exact infinite-horizon set (safe forever under actuator limits) is intractable for high-dimensional nonlinear systems, so GCBF+ approximates it with a **finite lookahead horizon** T (as $T \to \infty$ it recovers the true set).

Procedure. Roll out the *current* (on-policy, aligning train/test distributions) π_ϕ to get $\{z_i^0, \ldots, z_i^{\bar{T}}\}$, $\bar{T} \geq T$:

- z_i^t in collision $\Rightarrow \mathcal{D}_{\mathcal{A},i}$;
- $z_i^t, \ldots, z_i^{t+T}$ all collision-free $\Rightarrow \mathcal{D}_{\mathcal{C},i}$;
- otherwise (safe now, crashes within T steps) $\Rightarrow$ *unlabelled*.

Improvement over GCBFv0/MDCBF (Remark 6). Those forced $h > 0$ on the whole collision-free set $S_{N,i}$, ignoring momentum and control limits. If $S_{N,i}$ is not control-invariant, *no* valid CBF is positive on it, so this creates contradictions that hurt training. Margin/shrunk-set fixes fail for high relative degree (the margin is state-dependent).

8 Implementation Details

MLP	Role	Hidden layers	Output dim
ψ_1	Encoder	256×256	128
ψ_2	Attention gate (w_{ij})	128×128	1
ψ_3	Attention value	256×256	128
ψ_4	Readout	256×256	1 for h_θ; m for π_ϕ

- **Footprint:** trained policy takes 1.5 MB. Swish activations.
- **Residual trick:** $\pi_\phi = \pi_\phi^{NN} + \pi_{\text{nom}}$, so the network learns only the safety deviation from the goal-reaching controller.

- **Training:** Adam, 1000 steps, $\approx 3\,\text{h}$ on i7-13700KF + RTX 3090; data from 8 agents and 8 obstacles in area $l = 4$ (2D) or $l = 2$ (3D).
- **Fixed hyperparameters (all 5 envs):** $\alpha = 1.0$, $\gamma = 0.02$, $\eta_{\text{deriv}} = 0.2$, $R = 0.5\,\text{m}$, $r = 0.05\,\text{m}$.

Environment	Dynamics	T	η_{ctrl}	Policy LR	GCBF LR
SingleIntegrator	2D linear (velocity)	1	10^{-4}	10^{-5}	10^{-5}
DoubleIntegrator	2D linear (accel. limits)	32	10^{-4}	10^{-5}	10^{-5}
DubinsCar	2D nonlinear (unicycle)	32	10^{-5}	3×10^{-5}	3×10^{-5}
LinearDrone	3D linear quadrotor	32	10^{-3}	10^{-5}	10^{-5}
CrazyflieDrone	3D 12-state 6-DOF quadrotor	32	3×10^{-5}	10^{-5}	10^{-4}

9 Evaluation Metrics

Over 96 runs (3 seeds $\times$ 32 random test cases); agents keep moving after collisions (no collision dynamics):

- **Safety rate:** fraction of agents never colliding with obstacles or other agents.
- **Reaching rate:** fraction of agents reaching their goal by the end of the horizon.
- **Success rate:** fraction that are both safe *and* reach their goal.

10 Improvement Ideas

1. The analysis assumes no stochasticity $\Rightarrow$ **stochastic GCBF+**.
2. **A*/RRT** within the detection range as a high-level planner (the paper also mentions a planner to resolve deadlocks).
3. Some **game-theoretic** strategy or **MARL** methods to add cooperation among agents (the paper lists lack of cooperation as a cause of conservatism and deadlocks).